

RuralCare Project File Map

Project path: D:\ruralcare

This guide lists the important files, what code/config each contains, and what you can safely edit.

Main Editable Files

File	What is inside / why it matters
lib/main.dart	Main editable Flutter app file. Contains all RuralCare UI screens and demo logic: splash, language/role selection, admin login/dashboard, staff login/tasks, patient registration/login/dashboard, Firestore CRUD, request workflow, notifications, seeding, and logout handling.
pubspec.yaml	Flutter package manifest. Lists app name, SDK version, Firebase packages, intl, shared_preferences, and Flutter lint/test dependencies.
pubspec.lock	Generated dependency lock file created by flutter pub get. Do not edit manually unless resolving dependency issues.
analysis_options.yaml	Dart analyzer/lint configuration. Some demo-friendly lints are ignored so flutter analyze stays clean.
firebase.json	Firebase project configuration for Flutter/Firebase tooling. Points Firestore deploys to firestore.rules and firestore.indexes.json. No Cloud Functions are configured.
firestore.rules	Spark-plan demo Firestore security rules. Allows demo credential lookup for adminAccounts/staff, public service reads/seeding, and signed-in anonymous/phone-auth sessions for app data.
firestore.indexes.json	Firestore composite indexes used by request, notification, and other queries. Deploy with firebase deploy --only firestore:indexes if indexes change.
.firebaserc	Firebase CLI project alias. Current default points to ruralcare-production.
android/app/google-services.json	Android Firebase app configuration downloaded/generated by Firebase. Required for Android builds to connect to your Firebase project.
docs/Firebase_Setup_Guide.md	Markdown setup guide for Firebase Spark-plan demo configuration and collection names.
docs/Firebase_Setup_Guide.pdf	PDF version of the Firebase setup guide.
docs/Project_File_Map.md	Markdown version of this file guide.
docs/Project_File_Map.pdf	This PDF file guide.
test/widget_test.dart	Minimal Flutter test harness. Keeps the test folder valid; app verification currently uses flutter analyze.
README.md	Default project readme. Can be edited with your project overview if needed.

lib/main.dart Code Map

The app code is centralized here. These are the key classes/functions detected with line numbers.

Line 14: AppSession ? class AppSession {

Line 40: ensureFirebaseAuthSession ? Future<void> ensureFirebaseAuthSession() async {

Line 46: logAttempt ? Future<void> logAttempt(

Line 47: role ? String role,

Line 48: username ? String username,

Line 50: reason ? String reason,

Line 61: seedDefaultAdminIfMissing ? Future<void> seedDefaultAdminIfMissing() async {

Line 81: seedDefaultServicesIfMissing ? Future<void> seedDefaultServicesIfMissing() async {

Line 106: adminLoginDirect ? Future<Map<String, dynamic>> adminLoginDirect(

Line 107: username ? String username,

Line 108: password ? String password,

Line 155: resetAdminPasswordDirect ? Future<void> resetAdminPasswordDirect(

Line 156: username ? String username,

Line 157: answer ? String answer,

Line 158: newPassword ? String newPassword,

Line 178: staffLoginDirect ? Future<Map<String, dynamic>> staffLoginDirect(

Line 179: username ? String username,

Line 180: password ? String password,

Line 203: saveStaffDirect ? Future<void> saveStaffDirect(Map<String, String> data, {String? id}) async {

Line 246: deleteStaffDirect ? Future<void> deleteStaffDirect(String uid) async {

Line 255: saveServiceDirect ? Future<void> saveServiceDirect({

Line 282: notifyUser ? Future<void> notifyUser(

Line 283: uid ? String uid,

Line 284: role ? String role,

Line 285: title ? String title,

Line 286: body ? String body,

Line 287: requestId ? String requestId,

Line 300: createRequestDirect ? Future<void> createRequestDirect(Map<String, dynamic> payload) async {

Line 350: acceptRequestDirect ? Future<void> acceptRequestDirect(String requestId) async {

Line 383: rejectRequestDirect ? Future<void> rejectRequestDirect(String requestId, String reason) async {

Line 406: proposeTimeDirect ? Future<void> proposeTimeDirect(

Line 407: requestId ? String requestId,

Line 409: message ? String message,

Line 432: completeRequestDirect ? Future<void> completeRequestDirect(String requestId) async {

Line 453: main ? Future<void> main() async {

Line 461: RuralCareApp ? class RuralCareApp extends StatefulWidget {

Line 467: _RuralCareAppState ? class _RuralCareAppState extends State<RuralCareApp> {

Line 470: initState ? void initState() {

Line 477: setLang ? Future<void> setLang(String code) async {

Line 499: AppScope ? class AppScope extends InheritedWidget {

Line 550: t ? String t(BuildContext c, String k) =>

Line 552: when ? String when(dynamic v) {

Line 559: activePatientUid ? String activePatientUid() =>

Line 564: dec ? InputDecoration dec(String label) => InputDecoration(

Line 570: page ? Route page(Widget w) => MaterialPageRoute(builder: (_) => w);

Line 571: toast ? void toast(BuildContext c, String s) =>

Line 574: friendlyError ? String friendlyError(Object error) {

Line 583: runBusy ? Future<T?> runBusy<T>(BuildContext c, Future<T> Function() task) async {

Line 601: SplashScreen ? class SplashScreen extends StatelessWidget {

Line 646: LanguageScreen ? class LanguageScreen extends StatelessWidget {

Line 660: lang ? Widget lang(BuildContext c, String label, String code) => Card(

Line 672: RoleScreen ? class RoleScreen extends StatelessWidget {

Line 686: role ? Widget role(BuildContext c, IconData icon, String text, Widget next) => Card(

Line 696: AdminLogin ? class AdminLogin extends StatefulWidget {

Line 702: _AdminLoginState ? class _AdminLoginState extends State<AdminLogin> {

Line 705: dispose ? void dispose() {

Line 711: login ? Future<void> login() async {

Line 743: resetAdminDialog ? Future<void> resetAdminDialog(BuildContext c, String username) async {

Line 783: StaffLogin ? class StaffLogin extends StatefulWidget {

Line 789: _StaffLoginState ? class _StaffLoginState extends State<StaffLogin> {

Line 792: dispose ? void dispose() {

Line 798: login ? Future<void> login() async {

Line 827: PatientEntry ? class PatientEntry extends StatelessWidget {

Line 852: PatientLogin ? class PatientLogin extends StatefulWidget {

Line 858: _PatientLoginState ? class _PatientLoginState extends State<PatientLogin> {

Line 860: get ? String get mobileDigits => phone.text.trim().replaceAll(RegExp(r'D'), "");

Line 861: get ? String get e164 => '+91\$mobileDigits';

Line 863: dispose ? void dispose() {

Line 869: login ? Future<void> login() async {

Line 921: PatientRegister ? class PatientRegister extends StatefulWidget {

Line 927: _PatientRegisterState ? class _PatientRegisterState extends State<PatientRegister> {

Line 936: gender ? String gender = 'Male', blood = 'O+';

Line 937: get ? String get mobileDigits => mob.text.trim().replaceAll(RegExp(r'D'), "");

Line 938: get ? String get e164 => '+91\$mobileDigits';

Line 940: dispose ? void dispose() {

Line 968: register ? Future<void> register() async {

Line 1074: AdminHome ? class AdminHome extends StatefulWidget {

Line 1080: _AdminHomeState ? class _AdminHomeState extends State<AdminHome> {

Line 1109: StaffHome ? class StaffHome extends StatefulWidget {

Line 1116: _StaffHomeState ? class _StaffHomeState extends State<StaffHome> {

Line 1138: PatientHome ? class PatientHome extends StatefulWidget {

Line 1144: _PatientHomeState ? class _PatientHomeState extends State<PatientHome> {

Line 1161: logoutDirect ? Future<void> logoutDirect(BuildContext context) async {

Line 1174: shell ? Widget shell(

Line 1175: title ? String title,

Line 1200: AdminSummary ? class AdminSummary extends StatelessWidget {

Line 1236: count ? Widget count(String label, Stream<QuerySnapshot> stream) =>

Line 1263: AdminPatients ? class AdminPatients extends StatelessWidget {

Line 1291: RequestHistory ? class RequestHistory extends StatelessWidget {

Line 1309: AdminStaff ? class AdminStaff extends StatelessWidget {

Line 1354: staffDialog ? Future<void> staffDialog(

Line 1425: AdminServices ? class AdminServices extends StatelessWidget {

Line 1462: serviceDialog ? Future<void> serviceDialog(

Line 1520: StaffIncoming ? class StaffIncoming extends StatelessWidget {

Line 1539: StaffTasks ? class StaffTasks extends StatelessWidget {

Line 1556: RequestMonitor ? class RequestMonitor extends StatelessWidget {

Line 1571: requestCard ? Widget requestCard(

Line 1615: requestTile ? Widget requestTile(Map<String, dynamic> x) => ListTile(

Line 1621: rejectDialog ? Future<void> rejectDialog(BuildContext c, String id) async {

Line 1654: proposeDialog ? Future<void> proposeDialog(BuildContext c, String id) async {

Line 1718: BookService ? class BookService extends StatefulWidget {

Line 1724: _BookServiceState ? class _BookServiceState extends State<BookService> {

Line 1725: forWhom ? String forWhom = 'Self';

Line 1731: dispose ? void dispose() {

Line 1737: submit ? Future<void> submit() async {

Line 1872: PatientRequests ? class PatientRequests extends StatelessWidget {

Line 1891: PatientProfile ? class PatientProfile extends StatelessWidget {

Line 1933: patientEdit ? Future<void> patientEdit(BuildContext c, Map<String, dynamic> x) async {

Line 1986: StaffProfile ? class StaffProfile extends StatelessWidget {

Line 2008: Notifications ? class Notifications extends StatelessWidget {

Line 2051: DocList ? class DocList extends StatelessWidget {

Line 2082: line ? Widget line(Map<String, dynamic> x, String label, String key) => ListTile(

Line 2087: dropdown ? Widget dropdown(

Line 2088: label ? String label,

Line 2089: value ? String value,

Line 2100: form ? Widget form(BuildContext c, String title, List<Widget> kids) => Scaffold(

Generated / Platform File Groups

Folder / File Group	What it contains
android/	Android build project generated by Flutter. Important editable files are android/app/build.gradle.kts, android/app/src/main/AndroidManifest.xml, and android/app/google-services.json. Most Gradle wrapper, mipmap icons, generated registrants, crash logs, and Kotlin cache files should not be edited manually.
ios/	iOS build project generated by Flutter. Contains Xcode project, plist files, app icons, launch screens, and generated plugin registrants. Edit only if you are configuring iOS-specific permissions or Firebase plist files.
web/	Flutter web shell files: index.html, manifest.json, favicon, and app icons. Edit if you need web metadata or icons.
windows/	Windows desktop Flutter runner files generated by Flutter/CMake. Usually leave unchanged unless building a Windows-specific desktop app.
linux/	Linux desktop Flutter runner files generated by Flutter/CMake. Usually leave unchanged.
macos/	macOS desktop Flutter runner files generated by Flutter/Xcode. Usually leave unchanged unless configuring macOS permissions/signing.
.idea/	VS Code/Android Studio/IntelliJ project metadata. Usually not part of app logic.
.metadata, .flutter-plugins-dependencies, *.iml	Flutter/IDE metadata generated by tooling. Do not edit manually.
ruralcare/	Nested duplicate Flutter project copy currently mirroring the root app. The main working project is D:\ruralcare; edit root files first. The nested copy exists because earlier work synced it so root flutter analyze stayed clean.

Firestore Collections Used

adminAccounts, staff, patients, services, requests, notifications, loginAttempts

Editing Advice

- Edit lib/main.dart for screens, workflows, validation, Firebase reads/writes, and navigation.
- Edit firestore.rules for permission behavior, then deploy with firebase deploy --only firestore:rules.
- Edit pubspec.yaml only when adding/removing packages, then run flutter pub get.
- Avoid editing .dart_tool, build, generated plugin registrants, Gradle wrapper files, or platform caches.
- If Firebase project settings change, regenerate android/app/google-services.json using Firebase/FlutterFire tooling.